林炜翔 12632766 技术路线

#1.Flowchart

根据图用 IF，else 嵌套判断

```
a = 9 > b = 5 > c = 3
```

#2. Matrix multiplication

M1,M2 用 random 来生成

分三层 for 来实现矩阵乘法，一层控制行，一层控制列，最内层用于第 i 行 j 列的单个元素的计算

```
[[6 9 8 1 9 9 1 6 6 5]
 [3 1 7 0 5 6 9 3 3 5]
 [4 1 2 6 2 6 5 9 3 2]
 [0 5 0 8 2 0 4 8 6 7]
 [3 8 0 5 7 0 5 9 1 1]]
[[0 2 4 6 2]
 [0 3 0 9 5]
 [4 0 7 6 9]
 [7 7 9 9 8]
 [5 2 0 4 3]
 [1 0 6 7 7]
 [9 3 7 6 8]
 [2 1 8 6 0]
 [8 5 8 0 8]
 [2 6 2 8 0]]
[[172. 133. 256. 355. 283.]
 [180.  94. 218. 243. 227.]
 [157. 108. 255. 249. 191.]
 [180. 167. 226. 253. 175.]
 [143. 114. 174. 255. 155.]]
```

#3. Pascal triangle

先找到杨辉三角的样式确定基础计算逻辑，然后以列表形式，根据上一行的元素计算出下一行的对应位置的元素然后插入到代表下一行的列表中

以第 100 行为例

```
tell me the figure(>=2) : 100
[1, 100, 4950, 161700, 3921225, 75287520, 1192052400, 16007560800, 186087894300, 1902231808400, 17310309456440, 1416298046
43600, 1050421051106700, 7110542499799200, 44186942677323600, 253338471349988640, 1345860629046814650, 665013487293720180
0, 3066451080298820830 0, 13234157293921226740 0, 53598337040380968297 0, 20418141106213212560 0, 73320668851776562692 00, 248
65270306254660391200, 797760755659003687551 00, 24251926972033712101 5504, 69957481650097246446 7800, 19173532007804430507636
00, 49988137020347265252051 00, 12410847811948286545336800, 29372339821610944823963760, 66324638306863423796047200, 1430125
01349174257560226775, 29469242702254089436652 7900, 58071742972088940948698 1450, 10950671531879628864611650 20, 197720458214
4932989443770175, 34200295474939381439027375 00, 56700489866346869227861176 00, 90139240300346304926343408 00, 13746234145802
811501267369720, 20116440213369968050635175200, 28258808871162574166368460400, 38116532895986727945334202400, 493782357970
7371574736476 2200, 61448471214136179596720592960, 73470998190814997343905056800, 84413487283064039501507937600, 9320655887
50498769495816811 00, 98913082887808032681188722800, 10089134454556419333481249 7256, 98913082887808032681188722800, 9320655
88750498769495816811 00, 84413487283064039501507937600, 73470998190814997343905056800, 61448471214136179596720592960, 49378
23579707371574736476 2200, 38116532895986727945334202400, 28258808871162574166368460400, 20116440213369968050635175200, 137
46234145802811501267369720, 90139240300346304926343408 00, 56700489866346869227861176 00, 34200295474939381439027376 00, 1977
20458214493298944377017 5, 10950671531879628864611650 20, 58071742972088940948698 1450, 29469242702254089436652 7900, 14301250
134917425756022677 5, 66324638306863423796047200, 29372339821610944823963760, 12410847811948286545336800, 49988137020347265
25205100, 19173532007804430507636 00, 69957481650097246446 7800, 24251926972033712101 5504, 79776075565900368755100, 24865270
306254660391200, 73320668851776562692 00, 20418141106213212560 0, 53598337040380968297 0, 13234157293921226740 0, 30664510802
988208300, 6650134872937201800, 1345860629046814650, 25333847134998864 0, 44186942677323600, 7110542499799200, 105042105110
6700, 141629804643600, 17310309456440, 1902231808400, 186087894300, 16007560800, 1192052400, 75287520, 3921225, 161700, 49
50, 100, 1]
```

#4. Add or double

从结果倒推最小步数，即将 1 如何通过最短路径到 x 的过程变成 x 如何通过最短路径变成 1。运算逻辑上比正推更不容易多算步数。

```
x=71
at least move 9 times
```

#5. Dynamic programming

用 itertools.product 生成所有加减空组合，从 1 开始，依次把符号和下一个数字拼上去。如果 calculation 为空，两个数字就直接连在一起，形成多位数。之后用 eval 求值，之后比较判断输出。

5.2 是算出所有可能的算式后分别计算各结果的可能性出现的次数，储存在列表 l 里最后用 max()和 min()判断可能最多和最少，再根据对应的索引得知是哪个数字

```
1+2+3+4-56+7+89=50
1+2+3-4+56-7+8-9=50
1+2+34-5-6+7+8+9=50
1+2+34-56+78-9=50
1+2-3+4+56+7-8-9=50
1+2-34+5-6-7+89=50
1-2+3-45+6+78+9=50
1-2+34+5+6+7+8-9=50
1-2+34-5-67+89=50
1-2-3+4+56-7-8+9=50
1-2-3-4-5-6+78-9=50
1-2-34-5-6+7+89=50
1-23+4+5-6+78-9=50
1-23-4-5-6+78+9=50
12+3+4-56+78+9=50
12-3+45+6+7-8-9=50
12-3-4-5+67-8-9=50
最大值： 26 对应数字： 1
最小值： 6 对应数字： 88
```